Week 10 – Deadlocks

Operating Systems Ch 8

Today's Schedule

Hour	Content
1st	**Quiz (Beginning)** → Theory Lecture (Part 1)
2nd	Theory Lecture (Part 2)
3rd	Hands-on Lab

Part 1

System Model

Chapter Objectives

Topics covered in this chapter:

- Understand how deadlock arises when using mutex locks
- Define the 4 necessary conditions that characterize deadlock
- Identify deadlock situations in a Resource-Allocation Graph
- Evaluate 4 approaches for deadlock prevention
- Apply the Banker's Algorithm for deadlock avoidance
- Apply the deadlock detection algorithm
- Evaluate approaches for deadlock recovery

System Model

A system consists of a finite number of resources

- **Resource types**: R1, R2, …, Rm
  - CPU cycles, memory, I/O devices, files, mutex locks, semaphores, etc.
- Each resource type Ri has Wi **instances**
  - Example: If the system has 3 printers, R_printer has instances = 3

Resource Usage Sequence

The normal sequence for a thread using a resource:

1. **Request** – Request the resource (wait if not immediately available)
2. **Use** – Use the resource (print, read a file, etc.)
3. **Release** – Release the resource

System call examples:

Operation	System Calls
Device	`request()` / `release()`
File	`open()` / `close()`
Memory	`allocate()` / `free()`
Semaphore	`wait()` / `signal()`

Definition of Deadlock

Deadlock: A state in which every thread in a set is waiting for an event that can only be caused by another thread in the same set

```
Example:
  T1 holds resource A, waits for resource B
  T2 holds resource B, waits for resource A
  → Both threads can never make progress
```

Kansas law: "When two trains meet at a crossing, both shall come to a full stop, and neither shall start until the other has gone" – A logically impossible situation!

Deadlock in Multithreaded Applications

An example of acquiring two mutex locks in different orders:

```
pthread_mutex_t first_mutex;
pthread_mutex_t second_mutex;

/* thread_one */
void *do_work_one(void *param) {
    pthread_mutex_lock(&first_mutex);
    pthread_mutex_lock(&second_mutex);
    /* Do some work */
    pthread_mutex_unlock(&second_mutex);
    pthread_mutex_unlock(&first_mutex);
    pthread_exit(0);
}

/* thread_two */
void *do_work_two(void *param) {
    pthread_mutex_lock(&second_mutex);   // Opposite order!
    pthread_mutex_lock(&first_mutex);
    /* Do some work */
    pthread_mutex_unlock(&first_mutex);
    pthread_mutex_unlock(&second_mutex);
    pthread_exit(0);
}
```

Deadlock Occurrence Scenario

An execution sequence where deadlock occurs in the previous code:

```
Time 1: thread_one acquires first_mutex
        → first_mutex: locked by T1

Time 2: thread_two acquires second_mutex
        → second_mutex: locked by T2

Time 3: thread_one requests second_mutex → blocked!
        (held by T2)

Time 4: thread_two requests first_mutex → blocked!
        (held by T1)

Result: Both threads wait forever → Deadlock!
```

Note: Deadlock does not always occur – it depends on CPU scheduling

Livelock

Livelock: A state where threads continuously change state but make no actual progress

- Unlike deadlock, threads are not in a blocked state
- They keep retrying but never succeed

```
/* thread_one — using trylock */
void *do_work_one(void *param) {
    int done = 0;
    while (!done) {
        pthread_mutex_lock(&first_mutex);
        if (pthread_mutex_trylock(&second_mutex)) {
            /* Do some work */
            pthread_mutex_unlock(&second_mutex);
            pthread_mutex_unlock(&first_mutex);
            done = 1;
        } else
            pthread_mutex_unlock(&first_mutex);  // Release and retry
    }
    pthread_exit(0);
}
```

Solution: random backoff – randomize the retry interval (same as Ethernet collision resolution)

Part 2

Deadlock Characterization

Four Necessary Conditions for Deadlock

All four of the following conditions must hold **simultaneously** for deadlock to occur:

1. **Mutual Exclusion** – At least one resource is held in a nonsharable mode
2. **Hold and Wait** – A thread holds at least one resource while waiting for additional resources
3. **No Preemption** – Resources can only be released voluntarily by the thread holding them
4. **Circular Wait** – A circular chain of waiting exists: T0→T1→T2→…→Tn→T0

All four conditions must be satisfied for deadlock to occur; circular wait implies hold and wait

Four Necessary Conditions — Detailed Explanation

1. Mutual Exclusion — At least one resource is held in a nonsharable mode. Example: mutex lock

2. Hold and Wait — A thread holds at least one resource while requesting additional resources and waiting. Example: T1 holds disk and requests printer

3. No Preemption — Resources can only be released voluntarily by the thread holding them; forced reclamation is not possible. Example: mutex lock

4. Circular Wait — In {T0, T1, …, Tn}, a circular wait exists: T0→T1→…→Tn→T0

```
Circular Wait example:
  T0 ──(waits for R1)──→ T1
   ↑                       │
  (waits for R3)     (waits for R2)
   │                       ↓
  T2 ←──(holds R2)──────── T1
```

Resource-Allocation Graph (RAG)

Represents the resource allocation state of the system as a directed graph

Components:

- **Thread**: Circle (Ti)
- **Resource type**: Rectangle (Rj), dots inside = number of instances
- **Request edge**: Ti → Rj (thread requests a resource, waiting)
- **Assignment edge**: Rj → Ti (a resource instance is assigned to the thread)

```
Request edge:    Ti ──→ Rj   (Ti requests Rj)
Assignment edge: Rj ──→ Ti   (Rj is assigned to Ti)
```

RAG – Example (No Deadlock)

Represent the following state as a resource-allocation graph:

```
T = {T1, T2, T3}
R = {R1(1), R2(2), R3(1), R4(3)}

      R1            R3
       ●             ●
   ↗   ↓         ↓
(T1)   (T2)  ←———  (T3)
   ↖   ↑
      R2            R4
      ● ●           ●●●

T1: holds R2, waiting for R1
T2: holds R1, R2, waiting for R3
T3: holds R3
```

In this state, if T3 releases R3 then T2 can proceed → **No Deadlock**

RAG – Example (Deadlock)

If T3 additionally requests R2 from the previous state:

```
      R1            R3
       ●             ●
   ↗   ↓         ↓
(T1)  (T2)  ←———  (T3)
   ↖   ↑         ↑
       R2
       ● ●   T3→R2 (request)

Two cycles occur:
  T1 → R1 → T2 → R3 → T3 → R2 → T1
  T2 → R3 → T3 → R2 → T2

→ Deadlock!
```

RAG – Relationship Between Cycle and Deadlock

Cycle and Deadlock:

Situation	Conclusion
No cycle	No deadlock (necessary condition not met)
Cycle exists + each resource has 1 instance	**Definitely** deadlock
Cycle exists + resources have multiple instances	Deadlock is **possible** (but not guaranteed)

```
Example with cycle but no deadlock:

      ┌──┐
      R1
      ●
      └──┘
   ↗    ↓            If T4 releases R2
(T1)   (T3)          T3 can proceed
                     → cycle is broken
```

Part 3

Methods for Handling Deadlocks

Deadlock Handling Methods – 3 Approaches

1. **Deadlock Prevention** – Ensure that at least one of the 4 necessary conditions cannot hold
2. **Deadlock Avoidance** – Use additional information to prevent entering an unsafe state
3. **Deadlock Detection & Recovery** – Allow deadlock to occur, then detect and recover

Other:

- **Deadlock Ignorance (Ostrich Algorithm)** – Ignore deadlock
  - Adopted by most operating systems (Linux, Windows)
  - Cost-benefit: Since deadlock occurs rarely, the cost of prevention may be greater
  - Kernel/application developers handle it directly

Comparison of Deadlock Handling Methods

Method	Core Idea	Advantage	Disadvantage
Prevention	Eliminate necessary conditions	Guaranteed prevention	Reduced resource utilization/throughput
Avoidance	Maintain safe state	More flexible than prevention	Requires advance information, overhead
Detection & Recovery	Respond after occurrence	High resource utilization	Detection overhead, recovery cost
Ignorance	Ignore	Lowest overhead	Manual intervention needed on deadlock

In practice, systems may combine different methods for different resource types

Part 4

Deadlock Prevention

Prevention – Breaking Mutual Exclusion

Eliminating the Mutual Exclusion condition:

- Sharable resources do not require mutual exclusion
  - Example: Read-only files can be accessed simultaneously
  - If multiple threads only read, deadlock cannot occur
- However, resources like mutex locks and semaphores are **inherently nonsharable**
  - Cannot be applied to such resources

Conclusion: **Generally difficult to apply**

Prevention – Breaking Hold and Wait

Eliminating the Hold and Wait condition:

Method 1 – Request **all needed resources at once** before execution begins

Method 2 – Only request new resources when holding no resources at all

- Release all currently held resources before making new requests

```
Method 2 example:
  A thread reads a file from disk, sorts it, then prints via printer

  Phase 1: Request disk → read file → release disk
  Phase 2: Request printer → print → release printer

  (disk and printer are never held simultaneously)
```

Disadvantage: Low resource utilization, possible starvation

Prevention – Breaking No Preemption

Eliminating the No Preemption condition:

Protocol 1:

- If a thread's resource request is not immediately fulfilled, **release all currently held resources**
- Restart when both previous and new resources can be acquired

Protocol 2:

- If the requested resource is held by another **waiting thread**, preempt it from that thread
- If held by a non-waiting thread, the requesting thread waits

Applicable resources: Resources whose state can be easily saved/restored (CPU registers, memory, etc.)

Not applicable: Mutex locks, semaphores (difficult to save/restore state)

Prevention – Breaking Circular Wait

The most practical prevention technique!

Impose a **total ordering** on all resource types:

F: R → N (assign a unique integer to each resource)

Each thread must request resources **only in ascending order**

```
F(first_mutex) = 1
F(second_mutex) = 5

// Correct – ascending F values
lock(first_mutex);      // F = 1
lock(second_mutex);     // F = 5 (1 < 5, OK)

// Violation (prohibited)
lock(second_mutex);     // F = 5
lock(first_mutex);      // F = 1 (5 > 1, violation!)
```

Circular Wait Prevention – Proof

Proof (by contradiction):

Assume circular wait exists: T0 → T1 → … → Tn → T0

- Since Ti+1 holds Ri and requests Ri+1: F(Ri) < F(Ri+1)
- Therefore: F(R0) < F(R1) < … < F(Rn) < F(R0)
- This implies F(R0) < F(R0), which is a **contradiction**
- Therefore, circular wait cannot occur

Note:

- Defining a lock ordering alone is not sufficient – programmers **must adhere to it**
- In Java, `System.identityHashCode(Object)` is sometimes used as the ordering function

Pitfall of Lock Ordering – Dynamic Locks

When locks are determined dynamically, ordering becomes difficult:

```
void transaction(Account from, Account to, double amount) {
    mutex lock1, lock2;
    lock1 = get_lock(from);
    lock2 = get_lock(to);

    acquire(lock1);     // lock for 'from' account
    acquire(lock2);     // lock for 'to' account

    withdraw(from, amount);
    deposit(to, amount);

    release(lock2);
    release(lock1);
}
```

```
Thread A: transaction(checking, savings, 25.0)
          → lock(checking) → lock(savings)

Thread B: transaction(savings, checking, 50.0)
          → lock(savings) → lock(checking)

→ Deadlock possible!
```

Linux lockdep Tool

lockdep: A lock order verification tool in the Linux kernel

- Dynamically tracks lock acquire/release patterns during kernel execution
- Reports warnings when lock ordering violations are detected

Key detection capabilities:

1. **Order violation detection** – Reports when locks are acquired in incorrect order
2. **Interrupt handler deadlock detection** – Warns when a spinlock used in an interrupt handler is acquired without disabling interrupts

Characteristics:

- For development/testing environments only (disabled in production due to performance overhead)
- Since its introduction in 2006, deadlock reports decreased by 10x
- Recently applicable to user application Pthreads mutex locks as well

Prevention Methods Summary

Condition	Prevention Strategy	Practicality
Mutual Exclusion	Use sharable resources	Not applicable to most resources
Hold and Wait	Request all resources at once or request only when holding none	Low resource utilization, starvation
No Preemption	Force resource release	Only applicable to CPU registers, etc.
Circular Wait	Order resources, request in ascending order	**Most practical**

Part 5

Deadlock Avoidance

Deadlock Avoidance – Overview

Limitations of Prevention: Reduced resource utilization and system throughput

Idea behind Avoidance:

- Each thread declares in advance its **maximum resource requirement**
- The system considers the **future** when making allocation decisions
- Even if resources are available, allocation is denied if it leads to an unsafe state

Key concepts:

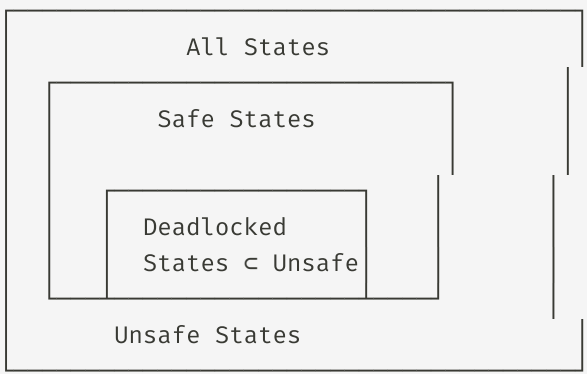
- **Safe State**: A state in which a sequence exists that allows all threads to complete
- **Unsafe State**: A state in which deadlock **may** occur

Safe State and Safe Sequence

Safe State: A state in which the system has a safe sequence

Safe Sequence <T1, T2, …, Tn>:

- Each Ti's resource demand can be satisfied by "currently available resources + resources to be returned by all Tj before Ti"
- That is, even if Ti's needed resources are not immediately available, they can be obtained once Tj (j < i) completes



Safe State Example

12 resources, 3 threads

Thread	Maximum Need	Current Allocation
T0	10	5
T1	4	2
T2	9	2

Available = 12 - (5 + 2 + 2) = **3**

```
Safe sequence: <T1, T0, T2>

  T1: Need = 4-2 = 2, Available = 3 ≥ 2
      → Execute and release → Available = 3+2 = 5

  T0: Need = 10-5 = 5, Available = 5 ≥ 5
```

Safe → Unsafe Transition Example

If we allocate 1 additional resource to T2 from the previous state:

Thread	Maximum Need	Current Allocation
T0	10	5
T1	4	2
T2	9	**3**

Available = 12 - (5 + 2 + 3) = **2**

```
Only T1 can run: Need = 2, Available = 2 ≥ 2
    → Execute and release → Available = 2+2 = 4

T0: Need = 5, Available = 4 < 5 → Cannot run!
T2: Need = 6, Available = 4 < 6 → Cannot run!
```

Resource-Allocation-Graph Algorithm

Used when each resource type has only **1 instance**

Add **Claim edge (dashed line)** to the existing RAG: Ti ⋯> Rj

- Claim edge: Ti **may** request Rj in the future (declared in advance)
- On resource request: Claim edge → Request edge
- On resource allocation: Request edge → Assignment edge
- On resource release: Assignment edge → Claim edge (restored)

Allocation rule: If converting a request edge to an assignment edge would create a **cycle, deny the allocation**

Cycle detection: O(n^2) – n is the number of threads

RAG Algorithm – Example

```
Initial state:

      ┌──────┐
      │  R1  │
      │  ●   │
      └──────┘

    ↗      \ (claim, dashed)
(T1)        (T2)
    \          ↗
 (claim)
           ┌──────┐
           │  R2  │
           │  ●   │
           └──────┘


What if T2 requests R2?

Converting to R2 → T2 (assignment):
T1 ···> R2 → T2 ···> R1 → T1  ← cycle formed!

→ Allocation denied! (would result in unsafe state)
```

If T1 requests R2 and T2 requests R1 → deadlock

Banker's Algorithm – Overview

The RAG Algorithm cannot be applied when resource instances are **more than one**

Banker's Algorithm:

- Based on the principle that a bank never lends cash in a way that it cannot satisfy all customers' demands
- Each thread declares its **maximum usage for each resource type** upon entering the system
- On resource request, check whether the system remains in a safe state after allocation
- If safe, allocate; if unsafe, wait

Banker's Algorithm – Data Structures

n = number of threads, m = number of resource types

```
Available[m]
  Available[j] = k → resource type Rj has k available instances

Max[n][m]
  Max[i][j] = k → thread Ti may request up to k instances of Rj

Allocation[n][m]
  Allocation[i][j] = k → Ti currently has k instances of Rj allocated

Need[n][m]
  Need[i][j] = Max[i][j] – Allocation[i][j]
  Additional instances of Rj that Ti may need
```

Vector comparison: X <= Y iff for all i, X[i] <= Y[i]

Safety Algorithm

Algorithm to determine whether the current state is safe:

```
1. Work[m] = Available
   Finish[n] = false (for all i)

2. Find an i such that:
   (a) Finish[i] == false
   (b) Need_i ≤ Work

   If no such i exists, go to step 4

3. Work = Work + Allocation_i
   Finish[i] = true
   Go back to step 2

4. If Finish[i] == true for all i → Safe State
   Otherwise → Unsafe State
```

Time complexity: O(m x n^2)

Resource-Request Algorithm

When thread Ti makes a request Request_i:

```
1. Check if Request_i ≤ Need_i
   If not → error (exceeds maximum claim)

2. Check if Request_i ≤ Available
   If not → Ti must wait (insufficient resources)

3. Perform pretend allocation:
   Available   = Available - Request_i
   Allocation_i = Allocation_i + Request_i
   Need_i      = Need_i - Request_i

4. Run Safety Algorithm:
   Safe   → Confirm allocation
   Unsafe → Cancel allocation, Ti waits
            (restore state before pretend allocation)
```

Banker's Algorithm – Example Initial State

5 threads (T0-T4), 3 resource types: A(10), B(5), C(7)

```
        Allocation    Max       Need
        A  B  C     A  B  C   A  B  C
T0      0  1  0     7  5  3   7  4  3
T1      2  0  0     3  2  2   1  2  2
T2      3  0  2     9  0  2   6  0  0
T3      2  1  1     2  2  2   0  1  1
T4      0  0  2     4  3  3   4  3  1

Total Allocated = (7, 2, 5)
Available = (10, 5, 7) - (7, 2, 5) = (3, 3, 2)
```

Need = Max - Allocation (additional resources each thread needs)

Banker's Algorithm – Safety Check Process

Work = (3, 3, 2), Finish = [F, F, F, F, F]

```
Step 1: Select T1 (Need(1,2,2) ≤ Work(3,3,2))
  Work = (3,3,2) + (2,0,0) = (5,3,2)
  Finish = [F, T, F, F, F]

Step 2: Select T3 (Need(0,1,1) ≤ Work(5,3,2))
  Work = (5,3,2) + (2,1,1) = (7,4,3)
  Finish = [F, T, F, T, F]

Step 3: Select T4 (Need(4,3,1) ≤ Work(7,4,3))
  Work = (7,4,3) + (0,0,2) = (7,4,5)
  Finish = [F, T, F, T, T]

Step 4: Select T2 (Need(6,0,0) ≤ Work(7,4,5))
  Work = (7,4,5) + (3,0,2) = (10,4,7)
  Finish = [F, T, T, T, T]

Step 5: Select T0 (Need(7,4,3) ≤ Work(10,4,7))
  Work = (10,4,7) + (0,1,0) = (10,5,7)
  Finish = [T, T, T, T, T]

Safe sequence: <T1, T3, T4, T2, T0> → Safe State!
```

Banker's Algorithm — Request Example

From the previous state, T1 requests Request_1 = (1, 0, 2)

```
Check 1: Request(1,0,2) ≤ Need[1](1,2,2)? → Yes
Check 2: Request(1,0,2) ≤ Available(3,3,2)? → Yes

Pretend allocation:
  Available    = (3,3,2) - (1,0,2) = (2,3,0)
  Allocation[1] = (2,0,0) + (1,0,2) = (3,0,2)
  Need[1]      = (1,2,2) - (1,0,2) = (0,2,0)
```

```
Safety Check (new state):
  T1: Need(0,2,0) ≤ (2,3,0) → Work = (5,3,2)
  T3: Need(0,1,1) ≤ (5,3,2) → Work = (7,4,3)
  T4: Need(4,3,1) ≤ (7,4,3) → Work = (7,4,5)
  T0: Need(7,4,3) ≤ (7,4,5) → Work = (7,5,5)
  T2: Need(6,0,0) ≤ (7,5,5) → Work = (10,5,7)

  Safe! → Allocation approved
  Safe sequence: <T1, T3, T4, T0, T2>
```

Banker's Algorithm – Denied Requests

After T1's request is approved, in the resulting state:

If T4 requests (3, 3, 0)?

- Request(3,3,0) <= Available(2,3,0)? → **No** (3 > 2)
- Insufficient resources, **must wait**

If T0 requests (0, 2, 0)?

- Request(0,2,0) <= Need0? → Yes
- Request(0,2,0) <= Available(2,3,0)? → Yes
- Pretend allocation: Available = (2,1,0), Allocation[0] = (0,3,0), Need[0] = (7,2,3)
- Safety check → **Unsafe!** → Allocation denied, T0 waits

Even if resources are available, allocation is denied if it leads to an unsafe state!

Part 6

Deadlock Detection

Deadlock Detection – Overview

When Prevention/Avoidance is not used:

- Allow deadlock to occur
- Periodically check for deadlock using a **detection algorithm**
- If deadlock is detected, perform **recovery**

Two cases:

1. Each resource type has **1 instance** → Wait-for Graph
2. Each resource type has **multiple instances** → Detection Algorithm

Detection – Single Instance (Wait-for Graph)

When each resource type has 1 instance:

Wait-for Graph:

- **Remove resource nodes** from the Resource-Allocation Graph
- Ti → Tj: Ti is waiting for a resource held by Tj
- **If a cycle exists → Deadlock**

```
Resource-Allocation Graph → Wait-for Graph

Ti → Rq → Tj  is converted to  Ti → Tj
```

Cycle detection: O(n^2) (n = number of threads)

Wait-for Graph – Example

Converting from a Resource-Allocation Graph to a Wait-for Graph:

```
Resource-Allocation Graph:          Wait-for Graph:

  (T1)──→[R1]──→(T2)                (T1)──→(T2)
           ↑                                │
  (T5)──→[R3]──→(T3)                (T5)──→(T3)
                  │                         │
  (T4)←─[R4]←──(T3)                (T4)←───(T3)
   │               ↑                 │
   └──→[R5]──→(T2)                   └──→(T2)
             └──→(T4)                      (T4)

Cycle in Wait-for Graph:
  T1 → T2 → T3 → T4 → T1  → Deadlock!
```

Detection – Multiple Instances

When each resource type has multiple instances:

Similar to Banker's Algorithm but with a key difference:

	Banker's (Avoidance)	Detection
Matrix used	**Max** (maximum demand)	**Request** (current requests)
Finish initialization	All false	True if Allocation_i == 0
Judgment criteria	Safe/Unsafe	Deadlocked/Not
Purpose	Prevent future deadlock	Diagnose current state

Detection Algorithm – Details

```
1. Work[m] = Available
   Finish[i] = false (if Allocation_i ≠ 0)
   Finish[i] = true  (if Allocation_i = 0)

2. Find i such that Finish[i] = false and Request_i ≤ Work
   If no such i exists, go to step 4

3. Work = Work + Allocation_i
   Finish[i] = true
   Go back to step 2

4. If Finish[i] = false for some thread Ti
   → Ti is deadlocked

   If all Finish[i] = true
   → No deadlock
```

Optimistic assumption: If a request is fulfilled, the thread will soon complete and return its resources

Detection Algorithm – Example (No Deadlock)

5 threads, 3 resource types: A(7), B(2), C(6)

```
         Allocation    Request     Available
         A  B  C      A  B  C     A  B  C
  T0     0  1  0      0  0  0     0  0  0
  T1     2  0  0      2  0  2
  T2     3  0  3      0  0  0
  T3     2  1  1      1  0  0
  T4     0  0  2      0  0  2
```

```
Step 1: Select T0 (Request(0,0,0) ≤ Work(0,0,0))
  Work = (0,0,0)+(0,1,0) = (0,1,0), Finish[0]=T

Step 2: Select T2 (Request(0,0,0) ≤ Work(0,1,0))
  Work = (0,1,0)+(3,0,3) = (3,1,3), Finish[2]=T

Step 3: Select T3 (Request(1,0,0) ≤ Work(3,1,3))
  Work = (3,1,3)+(2,1,1) = (5,2,4), Finish[3]=T

Step 4: Select T1 (Request(2,0,2) ≤ Work(5,2,4))
  Work = (5,2,4)+(2,0,0) = (7,2,4), Finish[1]=T

Step 5: Select T4 (Request(0,0,2) ≤ Work(7,2,4))
  Work = (7,2,4)+(0,0,2) = (7,2,6), Finish[4]=T
```

Detection Algorithm – Example (Deadlock)

From the previous state, T2 additionally requests 1 instance of C:

```
         Allocation    Request     Available
         A  B  C      A  B  C     A  B  C
  T0     0  1  0      0  0  0     0  0  0
  T1     2  0  0      2  0  2
  T2     3  0  3      0  0  1     ← changed!
  T3     2  1  1      1  0  0
  T4     0  0  2      0  0  2
```

```
Step 1: Select T0 (Request(0,0,0) ≤ Work(0,0,0))
  Work = (0,1,0), Finish[0]=T

Step 2: Search remaining threads for Request ≤ Work:
  T1: Request(2,0,2) ≤ (0,1,0)? No (2>0)
  T2: Request(0,0,1) ≤ (0,1,0)? No (1>0)
  T3: Request(1,0,0) ≤ (0,1,0)? No (1>0)
  T4: Request(0,0,2) ≤ (0,1,0)? No (2>0)

  → No further progress possible!

Finish = [T, F, F, F, F]
→ T1, T2, T3, T4 are Deadlocked!
```

Detection Algorithm Invocation Frequency

When should the detection algorithm be invoked?

Factors to consider:

1. How often does deadlock occur?
2. How many threads are affected by a deadlock?

Strategies:

- **On every request**: Immediate detection, can identify the causing thread, but high overhead
- **Periodically**: At fixed intervals (e.g., every hour)
- **Based on CPU utilization**: Invoke when CPU utilization drops below 40%
  - Deadlock reduces system throughput and lowers CPU utilization

Deadlock Management in Databases

Deadlock management in database systems:

- Deadlock can occur since locks are used during transaction processing
- Periodically search for cycles in the wait-for graph
- When deadlock is detected:
  1. **Select victim** – Choose the transaction with minimum cost
  2. **Abort transaction** – Roll back the victim's work
  3. **Release locks** – Return locks held by the victim
  4. **Continue remaining transactions** – Resolve the deadlock
  5. **Re-execute victim** – Restart the aborted transaction

MySQL selects the transaction with the fewest insert/update/delete rows as the victim

Part 7

Recovery from Deadlock

Recovery – Process Termination

Recovery action is needed when deadlock is detected

Method 1: Terminate all – Terminate **all** deadlocked processes

- Guaranteed resolution but expensive (discards long-running computations)

Method 2: Terminate one at a time – Terminate one at a time until the deadlock cycle is eliminated

- Requires re-running the detection algorithm each time → overhead
- Which process to terminate? (victim selection)

Victim Selection Criteria

Which process should be terminated (aborted) first?

Factors to consider:

1. **Priority** – Lower priority processes first
2. **Execution time** – How long has it been running, how much remains until completion
3. **Resources used** – Number and type of held resources (easy to preempt?)
4. **Resources needed** – Number of additional resources needed to complete
5. **Impact scope** – Number of processes affected by termination

Recovery – Resource Preemption

Forcibly reclaim resources from deadlocked processes

3 considerations:

1. Selecting a victim

- Reclaim resources from the process with minimum cost
- Use number of held resources, execution time, etc. as cost factors

2. Rollback

- Roll back the preempted process to a safe state
- Simple method: Total rollback (abort and restart)
- Advanced method: Roll back only as far as needed (requires checkpoints)

3. Starvation prevention

- Prevent the same process from being repeatedly selected as victim
- Include **rollback count in the cost factor** – cost increases with each rollback

Comparison of Recovery Methods

Method	Description	Advantage	Disadvantage
Terminate all	Kill all deadlocked processes	Guaranteed resolution	Expensive, computation loss
Terminate one at a time	Kill one by one until cycle breaks	Minimum cost possible	Requires repeated detection
Resource Preemption	Forcibly reclaim resources	Process preservation possible	Complex rollback, starvation risk

Real system approaches:

- Database: Transaction abort and rollback (natural recovery mechanism)
- OS: Mostly ignorance (manual reboot) or application-level handling

Lab

Banker's Algorithm Implementation

Lab – Banker's Algorithm Implementation (1)

Program structure

```
#include <stdio.h>
#include <stdbool.h>
#define N 5   // number of threads
#define M 3   // number of resource types

int available[M];
int max_res[N][M];
int allocation[N][M];
int need[N][M];        // need = max - allocation
```

Input: Available vector, Max matrix and Allocation matrix for each thread

```
// Compute the Need matrix
for (int i = 0; i < N; i++)
    for (int j = 0; j < M; j++)
        need[i][j] = max_res[i][j] - allocation[i][j];
```

Lab – Banker's Algorithm Implementation (2)

Safety Algorithm implementation

```
bool safety_algorithm(int safe_seq[]) {
    int work[M];
    bool finish[N];
    int count = 0;

    for (int j = 0; j < M; j++)
        work[j] = available[j];
    for (int i = 0; i < N; i++)
        finish[i] = false;

    while (count < N) {
        bool found = false;
        for (int i = 0; i < N; i++) {
            if (!finish[i]) {
                bool can_run = true;
                for (int j = 0; j < M; j++)
                    if (need[i][j] > work[j]) { can_run = false; break; }
                if (can_run) {
                    for (int j = 0; j < M; j++)
                        work[j] += allocation[i][j];
                    finish[i] = true;
                    safe_seq[count++] = i;
                    found = true;
```

Lab – Banker's Algorithm Implementation (3)

Resource Request simulation

```
bool request_resources(int tid, int request[]) {
    // 1. Check request ≤ need[tid]
    for (int j = 0; j < M; j++)
        if (request[j] > need[tid][j])
            return false;  // error: exceeds max claim

    // 2. Check request ≤ available
    for (int j = 0; j < M; j++)
        if (request[j] > available[j])
            return false;  // must wait

    // 3. Pretend allocation
    for (int j = 0; j < M; j++) {
        available[j] -= request[j];
        allocation[tid][j] += request[j];
        need[tid][j] -= request[j];
    }

    // 4. Safety check
    int safe_seq[N];
    if (safety_algorithm(safe_seq))
        return true;   // allocation approved
```

Lab – Main Function and Testing

```c
int main() {
    // Initialize with textbook example
    int avail[] = {3, 3, 2};
    int alloc[][3] = {{0,1,0},{2,0,0},{3,0,2},{2,1,1},{0,0,2}};
    int max_r[][3] = {{7,5,3},{3,2,2},{9,0,2},{2,2,2},{4,3,3}};

    for (int j = 0; j < M; j++) available[j] = avail[j];
    for (int i = 0; i < N; i++)
        for (int j = 0; j < M; j++) {
            allocation[i][j] = alloc[i][j];
            max_res[i][j] = max_r[i][j];
            need[i][j] = max_r[i][j] - alloc[i][j];
        }

    // Safety check
    int safe_seq[N];
    if (safety_algorithm(safe_seq)) {
        printf("Safe! Sequence: ");
        for (int i = 0; i < N; i++)
            printf("T%d ", safe_seq[i]);
        printf("\n");
    }

    // T1 requests (1, 0, 2)
```

Lab – Practice Assignments

Basic assignments:

1. Run the safety algorithm with the textbook example input and print the safe sequence
2. Simulate various requests and verify approval/denial
3. Intentionally create an unsafe state and identify which requests are denied

Extension assignments:

- Modify to dynamically accept the number of threads and resource types as input
- Process multiple requests sequentially and print the state at each step
- Also implement the Detection algorithm to check for deadlock occurrence

Summary

Key concepts this week:

Topic	Key Content
Deadlock definition	A state where all threads in a set are waiting for each other's events
Livelock	State changes but no progress (vs. Deadlock)
4 necessary conditions	Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait
Resource-Allocation Graph	Determine deadlock by cycle presence; depends on number of instances
Prevention	Break one condition (Circular Wait is most practical)
Avoidance	Maintain Safe State, Banker's Algorithm

Next Week Preview

Week 11 – Main Memory

- Basic Hardware, Address Binding
- Logical vs Physical Address Space
- Contiguous Memory Allocation (First-fit, Best-fit, Worst-fit)
- Paging (Page Table, TLB)
- Page Table Structures (Hierarchical, Hashed, Inverted)
- Swapping

Textbook: Ch 9